

Aspekte und Funktionale Programmierung

Inhalt

- Composition statt Vererbung
- Aspekte
- Funktionale Programmierung

Recall: Objektorientierte Programmierung

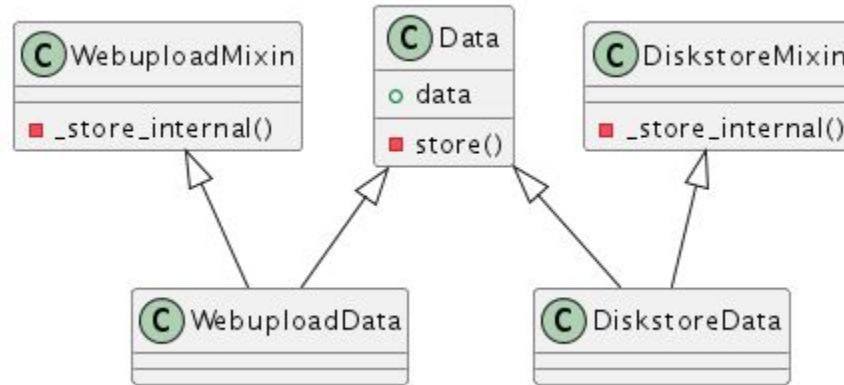
In der OOP benutzen wir Klassen und Vererbung um unseren Code zu strukturieren.

Wir vererben sowohl Dateneinträge als auch Funktionalität, diese können anschließend genutzt und verändert werden.

Das größte Problem der Vererbung ist, dass wir eine statische Abhängigkeit haben. Wir müssen, um jede neue Abhängigkeit zu modellieren, neue Klassen erzeugen.

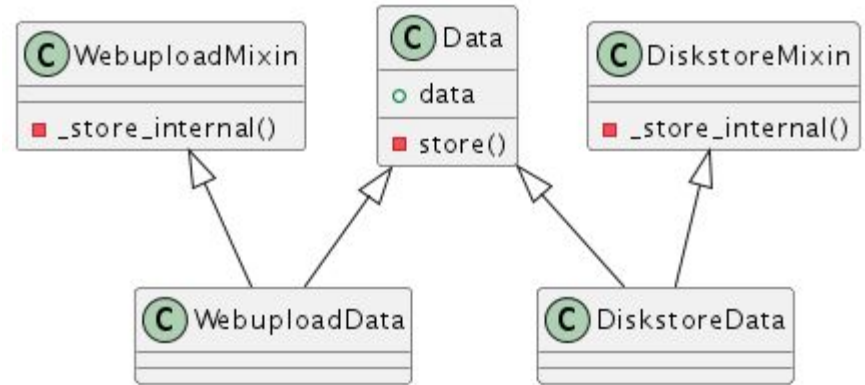
Beispiel Mixin-Klassen

Wir betrachten eine Klasse, die sowohl zum Webupload als auch zum Speichern auf die Festplatte benutzt werden kann.



Übersicht

- store in der Data Klasse ruft `_store_internal` auf.
- `_store_internal` wird von den Mixin Klassen definiert
- Je nach Mixin entsteht ein anderer Upload



Der Nachteil

Angenommen, wir wollten diese Klasse um mehrere Logging-Funktionalitäten erweitern:

- Console logging
- Cloud reporter logging

Nun könnten wir zwei weitere Mixins erstellen, und 4 abgeleitete Klassen erzeugen.

Diese Anzahl wächst exponentiell!

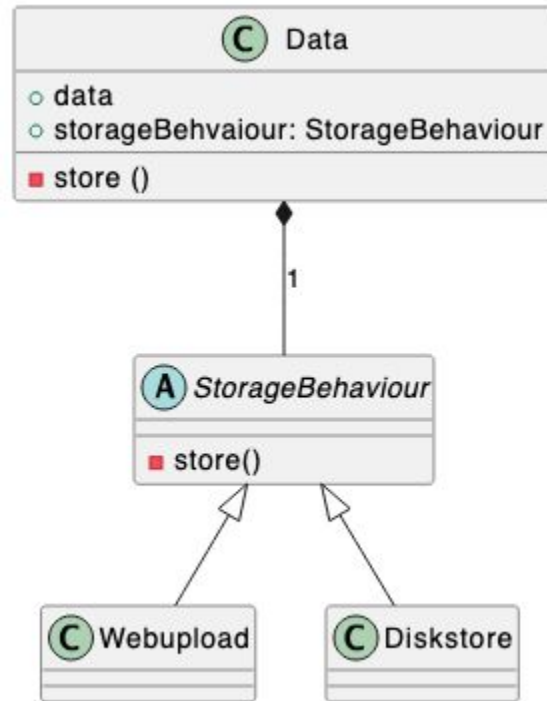
Composition

Statt weitere Mixins zu erzeugen, wird an einer solchen Stelle oft die Composition (Komposition) von Klassen verwendet.

Hierbei wird, anstatt Funktionalität in Klassen zu kapseln, diese mit Objekten gekapselt.

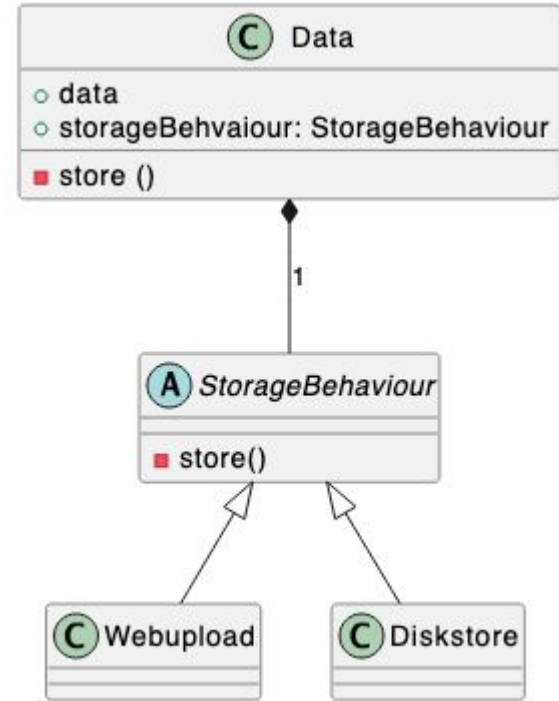
Die zugehörigen Klassen erben von einem Interface, welches diese dann benutzen können.

Composition

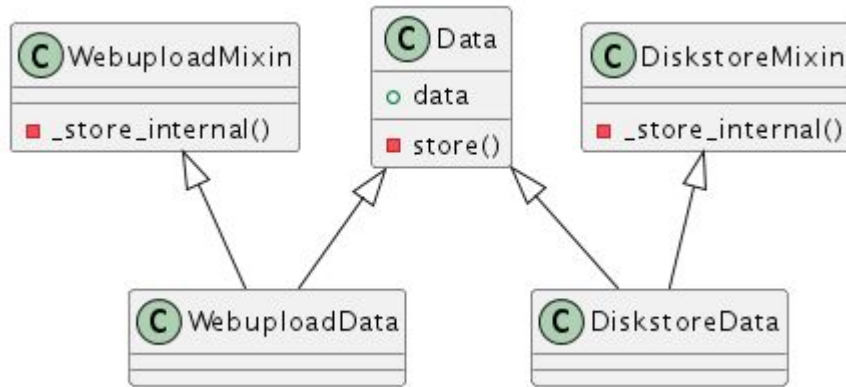


Vor- und Nachteile

- Da wir keine Vererbung vornehmen, können wir beliebig viele Compositions durchführen, ohne immer wieder neue Klassen zu erzeugen.
- Allerdings erzeugen wir zur Laufzeit mehr Objekte. Moderne Compiler können dies jedoch abstrahieren.
- Weiterhin kann es nicht zur Überlagerung von Datenelementen oder Funktionen kommen.



Im Vergleich



In python: Mixin

```
1  class Data:
2      def __init__(self, data):
3          self.data = data
4      def store(self):
5          self._store_internal(self.data)
6
7
8  class WebuploadMixin:
9      @staticmethod
10     def _store_internal(data):
11         print(f'Uploading {data} to web')
12
13
14     class DiskstoreMixin:
15         @staticmethod
16         def _store_internal(data):
17             print(f'Storing {data} to disk')
18
19
20     class WebuploadData(Data, WebuploadMixin):
21         pass
22
23
24     class DiskStoreData(Data, DiskstoreMixin):
25         pass
```

In python: Composition

```
class StorageBehaviour:
    @staticmethod
    def store(data):
        raise NotImplementedError()

class Webupload(StorageBehaviour):
    @staticmethod
    def store(data):
        print(f'Uploading {data} to web')

class Diskstore(StorageBehaviour):
    @staticmethod
    def store(data):
        print(f'Storing {data} to disk')

class Data:
    def __init__(self, data, storageBehaviour):
        self.data = data
        self.storageBehaviour = storageBehaviour
    def store(self):
        self.storageBehaviour.store(self.data)
```

Composition

Composition kann benutzt werden, um Verhalten einer Klasse zu ändern, jedoch nicht, um Verhalten hinzuzufügen.

Geht es nur um Änderung von Verhalten, so ist die Composition der Vererbung vorzuziehen. Wenn zusätzliches Verhalten erforderlich ist, so muss Vererbung angewendet werden.

Im Beispiel

Wir erinnern uns an Adas Blumenladen:

Es gab dort Kunden und Premiumkunden. Ein Premiumkunde kann eher nicht durch Composition aus einem Kunden entstehen, denn der Premiumkunde braucht zusätzliche Informationen.

Auf der anderen Seite gibt es Barzahlung und Kreditkartenzahlung. Ein Zahlungsmodul kann sicherlich die Art der Bezahlung durch Composition erhalten.

Aspekte

Aspekte stellen eine Erweiterung der Objektorientierung dar.

Oft ergeben sich Fälle, in denen bestimmte Funktionalitäten über mehrere Klassen/Methoden verwendet werden sollen. Beispiele hierfür sind Caching und Logging.

Ein Beispiel für einen Cache könnte so aussehen.

In python (ohne Aspekt)

```
class CachedArea:
    def __init__(self):
        self.cache = dict()
    def compute(self, a, b):
        if (a,b) in self.cache:
            return self.cache[(a,b)]
        else:
            return_val = a * b
            self.cache[(a,b)] = a * b
            return return_val
```

Aspekte

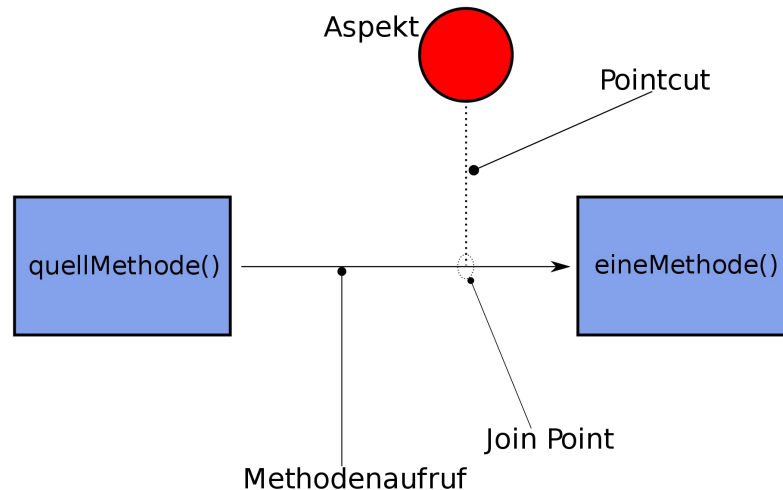
Man kann leicht feststellen, dass dieser Cachingansatz zu mehreren Problemen führt:

- Man muss den Cache immer wieder implementieren
- Man muss nachhalten, ob man die gleiche Instanz der Klasse `CachedArea` überall benutzt

```
class CachedArea:
    def __init__(self):
        self.cache = dict()
    def compute(self, a, b):
        if (a,b) in self.cache:
            return self.cache[(a,b)]
        else:
            return_val = a * b
            self.cache[(a,b)] = a * b
            return return_val
```

Aspekte

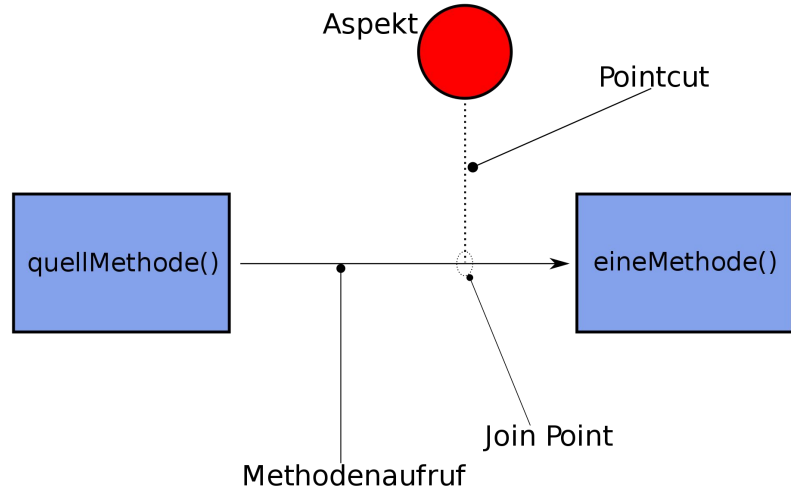
Um die vorher genannten Probleme zu vermeiden, werden globale Aspekte verwendet, um die gewünschte Funktionalität herzustellen. Hierzu wird eine Methode um einen Aspekt erweitert, welcher die Funktionalität erweitert. In einem Diagramm könnte dies so aussehen.



Aspekte

Join Point: Join points sind verschiedene vordefinierte Punkte, an denen ein Aspekt ausgeführt werden kann.

Point Cut: Der Point cut definiert, an welcher Stelle der Aspekt ausgeführt werden soll. Dies ist klassischerweise oft am vor oder Nach dem Methodenaufruf.



In der Realität

In der Realität gibt es wenig Sprachen, die das Konzept eines Aspekts voll abbilden. Eine hiervon ist AspectJ, die Java um das Konzept von Aspekten erweitert. Jedoch eignen sich auch viele Andere Sprachen für Aspekte.

Zuerst wollen wir uns Aspekte in AspectJ ansehen.

Die folgenden Code-Snippets wurden von Wikipedia übernommen.

In AspectJ

```
public void eineMethode() {  
    logger.trace("Betrete \"eineMethode\"");  
  
    // Abarbeitung der Methode  
    m = a + 2;  
  
    logger.trace("Verlasse \"eineMethode\"");  
}
```

In AspectJ

```
public aspect Tracing {  
    pointcut traceCall():  
        call(* AOPDemo.*(..));  
  
    before(): traceCall() {  
        System.out.println("Betrete \"" + thisJoinPoint + "\"");  
    }  
  
    after(): traceCall() {  
        System.out.println("Verlasse \"" + thisJoinPoint + "\"");  
    }  
  
}  
  
public void eineMethode() {  
    // Abarbeitung der Methode  
    m = a + 2;  
}
```

Zum Design

Da ein Aspekt im Normalfall äußerst transparent agiert, d.h., der Benutzer und der Entwickler bemerken ihn nicht, ist es schwierig ihn gut im Design zu dokumentieren.

Am besten ist es, den Aspekt selbst als eigene Komponente zu dokumentieren, und dann in jeder Klasse, die diesen Aspekt verwendet, darauf hinzuweisen.

Im Vorherigen Beispiel bedeutet dies: die Methode `eineMethode` benutzt den Aspekt `Tracing`.

In python

In python gibt es mehrere Möglichkeiten, Aspekte zu definieren. Eine davon sind decorators.

Decorators sind Funktionen, die zur Änderung einer Funktion bestimmt sind. Sie werden im Code vor eine Funktion geschrieben und können ihr Verhalten verändern.

Decorators in python

```
def logging_decorator(fun):  
    name = fun.__name__  
    def inner(*args, **kwargs):  
        print(f'starting {name}')  
        result = fun(*args, **kwargs)  
        print(f'finished {name} with result {result}')  
        return result  
    return inner  
  
@logging_decorator  
def compute(a,b):  
    return a * b
```

Decorators und Aspekte

Decorators können helfen, Aspekte zu transportieren. Hierbei sind sowohl Pointcuts als auch JoinPoints festgelegt: Diese befinden sich erst einmal vor oder nach der veränderten Methode.

Es ist auch in Python möglich, mehrere JoinPoints festzulegen. Hierzu können zum Beispiel Coroutinen verwendet werden.

In python

```
def adding_decorator(fun):  
    def inner():  
        generator = fun()  
        sum = 0  
        for i in generator:  
            sum = sum + i  
        return sum  
    return inner  
  
@adding_decorator  
def range_enumerator():  
    for i in range(10):  
        yield i
```

In python

In diesem Beispiel haben wir mehrere Join Points und PointCuts: Jede Iteration der Schleife in `range_enumerator` ist ein JoinPoint, und wird als PointCut benutzt.

```
def adding_decorator(fun):  
    def inner():  
        generator = fun()  
        sum = 0  
        for i in generator:  
            sum = sum + i  
        return sum  
    return inner  
  
@adding_decorator  
def range_enumerator():  
    for i in range(10):  
        yield i
```

Zurück zum Cache

```
def cache_decorator(fun):  
    cache = dict()  
    def inner(*args):  
        if args in cache:  
            return cache[args]  
        result = fun(*args)  
        cache[args] = result  
        return result  
    return inner  
  
@cache_decorator  
def factorial(value):  
    if value == 1:  
        return value  
    return value * factorial(value - 1)
```

In python

In diesem Fall wird der Cache vom Decorator initialisiert und dann um die Funktion herum gesetzt. Wir können beliebig viele Funktionen mit einem Cache ausstatten, jede enthält ihren eigenen Cache.

All dies funktioniert nun mit nur einer Zeile Code.

```
def cache_decorator(fun):  
    cache = dict()  
    def inner(*args):  
        if args in cache:  
            return cache[args]  
        result = fun(*args)  
        cache[args] = result  
        return result  
    return inner  
  
@cache_decorator  
def factorial(value):  
    if value == 1:  
        return value  
    return value * factorial(value - 1)
```

Weitere Möglichkeit: Introspection

Es ist auch ohne Decorator möglich, einen Caching Aspekt zu implementieren.

Hierzu kann man ausnutzen, dass man Klassenmethoden in Python beliebig verändern kann.

Wir rufen hierzu im Konstruktor eine Methode auf, die die Funktionen entsprechend anpasst.

In python

```
def cache(fun):
    cache = dict()
    def inner(*args):
        print('Caching')
        if args in cache:
            return cache[args]
        result = fun(*args)
        cache[args] = result
        return result
    return inner

class MethodCache:
    def __init__(self):
        for attribute in dir(self):
            if attribute.startswith('make'):
                setattr(self, attribute, cache(getattr(self, attribute)))

class Cached(MethodCache):
    def make_values(self, a, b):
        return a * b
    def values(self, a, b):
        return a * b
```

In python

Hier werden alle Methoden der abgeleiteten Klasse, die mit make beginnen. mit einem Cache ausgestattet. Der Vorgang ist vollständig ohne weiteren Code des Entwicklers möglich.

Der Entwickler der jeweiligen Klasse muss sich also keine weiteren Gedanken machen.

```
def cache(fun):
    cache = dict()
    def inner(*args):
        print('Caching')
        if args in cache:
            return cache[args]
        result = fun(*args)
        cache[args] = result
        return result
    return inner

class MethodCache:
    def __init__(self):
        for attribute in dir(self):
            if attribute.startswith('make'):
                setattr(self, attribute, cache(getattr(self, attribute)))

class Cached(MethodCache):
    def make_values(self, a, b):
        return a * b
    def values(self, a, b):
        return a * b
```

Funktionale Programmierung

Was ist funktionale Programmierung

Während bei der objektorientierten Programmierung Klassen und Objekte im Vordergrund stehen, ist das Hauptkonzept der funktionalen (funktionsorientierten) Programmierung die Funktion.

Funktionale Programmierung folgt einem strengen mathematischen Konzept. Wir wollen uns jedoch nicht mit der theoretischen Grundlage befassen, sondern mit dem, was wir in der Softwareentwicklung täglich nutzen können.

Die Funktion

Eine Funktion ordnet jedem Element x einer Grundmenge D genau ein Element y einer Zielmenge Z zu. Man schreibt:

$$f: D \rightarrow Z, x \mapsto y,$$

Wichtig ist hierbei das Konzept der Zuordnung: Jedem x wird ein unabhängiges y zugeordnet. Eine weitere Schreibweise für y ist $f(x)$.

Beispiel: Funktion

Die Funktion

$$f: \mathbb{N} \rightarrow \mathbb{N}, x \mapsto x^2,$$

ordnet einer natürlichen Zahl x ihr Quadrat zu.

Der Definitionsbereich sind die natürlichen Zahlen. Die Funktion ist also nur für natürliche Zahlen, nicht etwa für negative oder nicht ganzzahlige rationale Zahlen definiert, auch wenn die Abbildungsvorschrift erst einmal so aussieht.

Beispiel: Funktion

Die Einschränkung des Definitionsbereichs ist uns in der Programmierung geläufig. Wir betrachten die folgende Funktion (C++):

```
int square(int num) {  
    return num * num;  
}
```

Definitions- und Zielbereich sind durch den Typen `int` gekennzeichnet. Dieser besagt, dass eine ganze Zahl als Eingabe benutzt werden muss, und eine ganze Zahl als Rückgabe garantiert wird.

Unterschied Funktion und “normale” Methode

In der Programmierung ist es erlaubt, dass Methoden das Verhalten von Objekten, die sie als Eingabe erhalten, oder den globalen Kontext des Programms verändern. Man spricht von Seiteneffekten.

Ein klassisches Beispiel hierfür ist das Verändern globaler Variablen oder set-Methoden von Objekten.

In python: Seiteneffekte

```
class ChangeMe:
    def __init__(self, something):
        self.something = something
    def set_something(self, something):
        self.something = something

x = ChangeMe(5)
x.something # -> 5
x.set_something(6)
x.something # -> 6
```

In python: Seiteneffekte

```
glob = 5

def inc_glob():
    global glob
    glob = glob + 1

inc_glob()
glob # -> 6
```

Seiteneffekte

Seiteneffekte führen zu diversen Problemen im Programmablauf: Es ist nicht klar, welchen Effekt das Ausführen einer Methode hat, denn der Effekt hängt vom aktuellen Kontext ab.

In der funktionalen Programmierung wird daher auf Seiteneffekte verzichtet. Eine Methode hat bei gleicher Eingabe immer die gleiche Ausgabe. Auch wird kein globaler Kontext geändert. Weiterhin hängt das Ergebnis einer Methode nicht vom Kontext ab.

Von Seiteneffekten zu funktionaler Programmierung

Wir wollen nun sehen, wie wir die Beispiele von vorher in einen Seiteneffektfreien Kontext bringen können.

```
glob = 5

def inc_glob():
    global glob
    glob = glob + 1

inc_glob()
glob # -> 6
```

```
def inc(a):
    return a + 1

glob = 5
glob = inc(glob)
glob # -> 6
```

Von Seiteneffekten zu funktionaler Programmierung

- Anstatt die globale Variable direkt zu erhöhen, haben wir lediglich eine Funktion, die eine Variable erhöht.
- Diese rufen wir dann mit der globalen Variable als Argument auf und weisen das Ergebnis zu.
- Die Änderung des Kontext findet nun außerhalb der Funktion statt.
- Generell gilt: In der funktionalen Programmierung muss das Ergebnis einer Funktion immer verwendet werden.

```
def inc(a):  
    return a + 1  
  
glob = 5  
glob = inc(glob)  
glob # -> 6
```

Von Seiteneffekten zu funktionaler Programmierung

Eigentlich kommt die funktionale Programmierung ohne Klassen und Klassenmethoden aus. Es hat sich jedoch eingebürgert, diese weiterhin zu verwenden:

```
class ChangeMe:
    def __init__(self, something):
        self.something = something
    def add_something(self, something):
        return ChangeMe(self.something + something)

x = ChangeMe(5)
x.something # -> 5
x = x.set_something(6)
x.something # -> 11
```

Von Seiteneffekten zu funktionaler Programmierung

- Hier wird nicht der Wert `something` des Objektes der Klasse `ChangeMe` verändert, sondern eine neues Objekt der Klasse mit dem veränderten Wert erzeugt.
- Dies ist wichtig, denn der Input darf nicht verändert werden.

```
class ChangeMe:
    def __init__(self, something):
        self.something = something
    def add_something(self, something):
        return ChangeMe(self.something + something)

x = ChangeMe(5)
x.something # -> 5
x = x.set_something(6)
x.something # -> 11
```

Von Seiteneffekten zu funktionaler Programmierung

Es ist wichtig, den Input niemals zu verändern:

```
def increment_dict(input):  
    for key, value in input.items():  
        input[key] = input[key] + 1  
    return input # unnecessary  
  
def increment_dict(input):  
    output = dict()  
    for key, value in input.items():  
        output[key] = input[key] + 1  
    return output # input remains unchanged
```

Von Seiteneffekten zu funktionaler Programmierung

- Hier ändert die erste Version den Input, die zweite nicht.
- Generell sollte in der funktionalen Programmierung eine Eingabe nie als lvalue (left value, links der Zuweisung) verwendet werden.
- Um effizienter zu programmieren kann es aber hilfreich sein, zu verstehen, wie die verwendete Programmiersprache Eingaben verarbeitet.

```
def increment_dict(input):  
    for key, value in input.items():  
        input[key] = input[key] + 1  
    return input # unnecessary  
  
def increment_dict(input):  
    output = dict()  
    for key, value in input.items():  
        output[key] = input[key] + 1  
    return output # input remains unchanged
```

Beispiel: Call by Reference and Call By Value

- Die erste Funktion benutzt call by value (die Eingabe wird kopiert). Die Zuweisung ändert die Eingabe nicht.
- Die zweite Funktion benutzt call by reference (die Eingabe wird nicht kopiert). Die Zuweisung ändert die Eingabe.
- Nicht in jeder Sprache ist dieses Verhalten explizit. Man sollte sich also entweder informieren (Dokumentation, Test) oder aber davon ausgehen, dass call by reference verwendet wird.

```
// call by value
```

```
✓ int inc(int a){  
    a = a + 1;  
    return a;  
}
```

```
// call by reference
```

```
✓ int inc(int& a){  
    a = a + 1;  
    return a;  
}
```

Funktionale Programmierung

Da in der funktionalen Programmierung keine Seiteneffekte auftreten, kann jeder Programmablauf als Verkettung von Funktionen verstanden werden.

Eingaben werden von Funktionen so lange verarbeitet, bis das Endergebnis erhalten wird. Dies hat einige Vorteile:

- Einfache Parallelisierung
- Benutzung von serverless design
- Sehr einfache Erweiterbarkeit

Funktionalen Programmierung: Map und Reduce

In der funktionalen Programmierung sollte ein iterativer Programmaufbau vermieden werden. Stattdessen soll ein deklarativer Programmaufbau erzeugt werden.

Anstatt Objekte (Listen, etc.) nach und nach aufzubauen, benutzen wir Deklarationen, um diese Objekte aufzubauen.

Beispiel: Deklarativer Aufbau

```
# iterativ
def even_numbers(numbers):
    even_numbers = []
    for number in numbers:
        if number % 2 == 0:
            even_numbers.append(number)
    return even_numbers

# deklarativ
def even_numbers(numbers):
    return [number for number in numbers if number % 2 == 0]
```

Deklarativer Aufbau

- In der ersten Funktion wird die Liste `even_numbers` nach und nach aufgebaut. Da theoretisch jeder Schritt vom vorherigen abhängen kann, muss diese Funktion der Reihe nach ausgeführt werden.
- In der zweiten Funktion definieren wir die Liste `even_numbers`. Wir können die Bedingung für alle Einträge gleichzeitig untersuchen, denn der Computer weiß, dass kein Zugriff auf die bisher berechnete Liste möglich ist.

```
# iterativ
def even_numbers(numbers):
    even_numbers = []
    for number in numbers:
        if number % 2 == 0:
            even_numbers.append(number)
    return even_numbers

# deklarativ
def even_numbers(numbers):
    return [number for number in numbers if number % 2 == 0]
```

List comprehension

Das syntaktische Konstrukt

```
def even_numbers(numbers):  
    return [number for number in numbers if number % 2 == 0]
```

nennt man “List Comprehension”. Es erlaubt das deklarative Erzeugen einer neuen Liste, ohne hierbei einen Zwischenstand zu erzeugen. Dies ermöglicht dem Computer, die nötigen Operationen in beliebiger Reihenfolge auszuführen.

Listen und Iteratoren

Generell ist es wichtig, zwischen Listen und Iteratoren zu unterscheiden.

Eine Liste ist ein Objekt, das Einträge enthält, auf welche über einen Index zugegriffen werden kann. Ein Iterator ist ein Objekt, welches den nächsten Eintrag einer “Liste” zurückgeben kann. Oft kann es schneller sein, Iteratoren statt Listen zu verwenden.

Insbesondere lassen sich Iteratoren schachteln. Es muss erst etwas ausgerechnet werden, wenn wirklich ein Element benötigt wird.

Listen und Iteratoren

```
# Listen
square_list = [number * number for number in range(100000000)]
even_list = [number for number in square_list if number % 2 == 0]

# Iterator
square_iterator = iter(number * number for number in range(100000000))
even_list = [number for number in square_iterator if number % 2 == 0]

# Iteratoren
square_iterator = iter(number * number for number in range(100000000))
even_iterator = iter(number for number in square_iterator if number % 2 == 0)
```

Listen und Iteratoren

Im ersten Beispiel werden beide Listen komplett erzeugt. Dies ist langsam, denn die zweite Liste kann erst nach vollständiger Erzeugung der ersten erzeugt werden.

Im zweiten Beispiel wird nur noch die zweite Liste vollständig erzeugt.

Im dritten Beispiel wird keine Liste mehr vollständig erzeugt. Lediglich ein Iterator, der das Erzeugen der Liste erlaubt, wird zurückgegeben.

```
# Listen
square_list = [number * number for number in range(100000000)]
even_list = [number for number in square_list if number % 2 == 0]

# Iterator
square_iterator = iter(number * number for number in range(100000000))
even_list = [number for number in square_iterator if number % 2 == 0]

# Iteratoren
square_iterator = iter(number * number for number in range(100000000))
even_iterator = iter([number for number in square_iterator if number % 2 == 0])
```

Map und Reduce

Eine eingeschränkte Version der list comprehension ist das Konzept Map. Map wendet eine Funktion auf jeden Eintrag einer Liste an und gibt entweder eine Liste oder einen Iterator zurück.

Oft werden für diesen Aufruf anonyme Funktionen, so genannte Lambdas, verwendet.

Map und Reduce

```
xx = range(100)

def square(x):
    return x*x

xx_squared = [square(x) for x in xx]

# Alternative, using map
xx_squared = map(square, xx)

# Alternative, using lambda
xx_squared = map(lambda x: x*x, xx)
```

Warum map

List comprehensions und map verhalten sich in normalem Code genau gleich.

Der Unterschied ist hierbei unsichtbar: List comprehensions sind ein Syntax-Konstrukt, welches benutzt werden kann, um Code zu vereinfachen.

Map ist eine Funktion, die Abhängig vom Compute-Kontext überladen werden kann.

Dies bedeutet, dass ein map so konfiguriert werden kann, dass es auf dem Laptop normal ausgeführt werden kann und hoch parallelisiert auf einem großen Rechencluster. Da die benötigte Funktion direkt übergeben wird, ist dies einfach zu abstrahieren, also für den Nutzer unsichtbar zu implementieren.

Map und Reduce

Reduce ist der Gegenpart zu Map. Während Map eine Funktion auf jeden Eintrag einer Liste anwendet, sammelt Reduce die Einträge einer Liste.

```
from functools import reduce

def sum(iterable):
    sum = 0
    for i in iterable:
        sum = sum + i
    return sum

summed_entries = reduce(lambda x, i: x+i, xx_squared, 0)
```

Reduce

Reduce ersetzt eine Akkumulationsschleife. Wir übergeben hierbei eine Funktion, die einen Akkumulationsparameter übergeben bekommt. Dieser wird immer weitergegeben. Der zweite Parameter ist der jeweilige Parameter aus der Liste.

Die an Reduce übergebende 0 ist der Startwert für den Akkumulationsparameter.

```
from functools import reduce

def sum(iterable):
    sum = 0
    for i in iterable:
        sum = sum + i
    return sum

summed_entries = reduce(lambda x, i: x+i, xx_squared, 0)
```

Reduce

Mit der Benutzung von Reduce erlauben wir es dem Programm, die Liste in beliebiger Reihenfolge zu akkumulieren. Dies kann von Vorteil sein, wenn wir woher unterschiedliche Maps angewendet haben, und die Ergebnisse nicht in gleicher Reihenfolge im Reduce ankommen, wie sie in der ursprünglichen Liste vorhanden waren.

Dies bedeutet, die gegebene Funktion muss assoziativ sein: $(a+b)+c = a+(b+c)$.

Map und Reduce

Die Benutzung von Map und Reduce verschmilzt zu dem Verfahren MapReduce (Google).

Hierbei werden auf einem großen Datensatz verschiedene Maps angewendet. Dieses Verfahren ist wichtig für big data Anwendungen, z.B., machine learning, und zum Beispiel in Apache Spark oder in ElasticMapReduce implementiert.

Pipelines

Ein weiteres Konzept in der funktionalen Programmierung sind sogenannte Pipelines. Sie werden oft verwendet, um eine Reihe von Transformationen auf einem Eingabedatum auszuführen.

Eine Pipeline besteht aus einer Liste von Funktionen, die nacheinander auf jedes Objekt angewendet werden. Jede Funktion gibt ihre Ausgabe als Eingabe an die nächste Funktion.

Beispiel: Normalisierung eines Dictionaries

```
from copy import deepcopy

entries = [
    {'name': 'Spencer', 'gruss': 'hallo'}
]

def transform_all_entries(entries):
    for entry in entries:
        entry['sendung'] = 'sesamstraße'
        entry['sprache'] = 'deutsch'
        entry['name'] = entry['name'].lower()
```

```
def add_sendung(entry):
    entry = deepcopy(entry)
    entry['sendung'] = 'sesamstraße'
    return entry

def add_sprache(entry):
    entry = deepcopy(entry)
    entry['sprache'] = 'deutsch'
    return entry

def lowercase_name(entry):
    entry = deepcopy(entry)
    entry['name'] = entry['name'].lower()
    return entry

from functools import reduce

def pipeline_each(data, fns):
    return reduce(lambda a, x: map(x, a),
                  fns,
                  data)
```

Pipelines

Gegenüber dem iterativen Ansatz haben Pipelines einige Vorteile

- Sie können beliebig erweitert und wiederverwendet werden.
- Sie können verteilt angewendet werden: Es besteht keine Notwendigkeit, alle Daten auf einem Computer zu halten, oder alle Teile der Pipeline auf einem Computer anzuwenden.

```
def add_sendung(entry):  
    entry = deepcopy(entry)  
    entry['sendung'] = 'sesamstraße'  
    return entry  
  
def add_sprache(entry):  
    entry = deepcopy(entry)  
    entry['sprache'] = 'deutsch'  
    return entry  
  
def lowercase_name(entry):  
    entry = deepcopy(entry)  
    entry['name'] = entry['name'].lower()  
    return entry  
  
from functools import reduce  
  
def pipeline_each(data, fns):  
    return reduce(lambda a, x: map(x, a),  
                  fns,  
                  data)
```

Funktionales Design

Wir wollen nun verstehen, wie wir funktionale Ansätze in unserem Design verarbeiten können.

In der Objektorientierung bestand das Design im Wesentlichen aus der Problemanalyse und dem Softwaredesign. Das Softwaredesign wiederum basierte auf einem Klassendiagramm und auf Ablaufplänen.

Da es in der funktionalen Programmierung nicht auf den Stand von Objekten ankommt, wird das funktionale Softwaredesign in der Hauptsache aus Typen und Datenfluss bestehen. Die Problemanalyse bleibt erhalten.

Typen

In der funktionalen Programmierung werden die Objekte, welche die Daten enthalten, nicht über Klassen beschrieben. Vielmehr werden die Daten direkt beschrieben.

Diese Beschreibung und Kategorisierung der Daten bilden die Typen.

In einem funktionalen Design müssen zuerst die beteiligten Datentypen beschrieben werden.

(Primitive) Typen

Ein Typ beschreibt eine Menge von Objekten. Man unterscheidet zwischen primitiven Typen, zusammengesetzten Typen, und Funktionstypen

Primitive Typen sind die Basiskonzepte einer Programmiersprache. Beispiele (die in eigentlich jeder Programmiersprache vorhanden sind):

- Der Typ der ganzen Zahlen zwischen -2,147,483,648 und 2,147,483,647, `int32`
- Der Typ der Zeichenketten: `string`
- Der Typ der Charaktere: `char`

Wir nennen die eigentlichen Objekte Werte und sagen: Ein Wert ist vom Typ X.

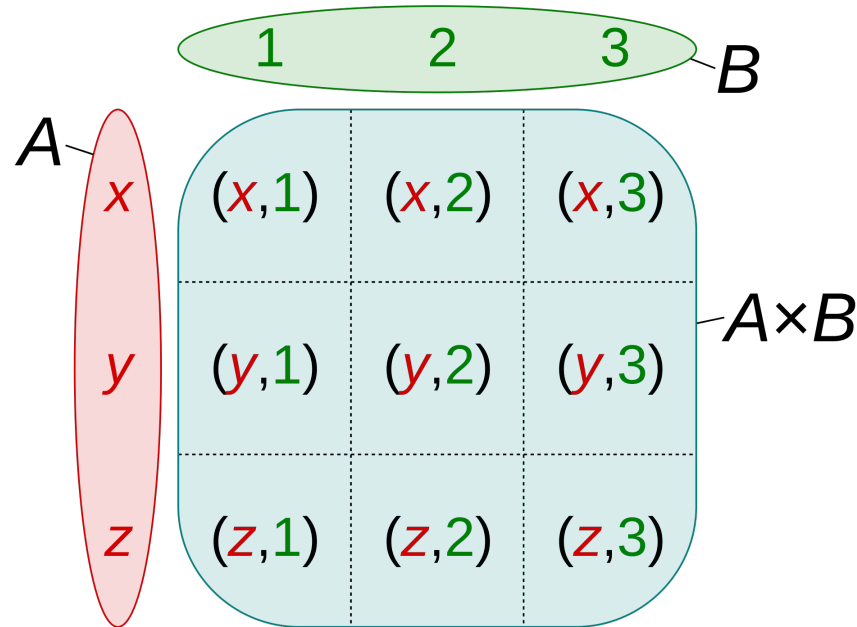
Typen und Mengen

Man könnte denken, dass Typen mathematische Mengen sind. Eine Menge enthält jedoch ihre Objekte. Dies ist bei einem Typ nicht der Fall. Vielmehr hat ein Objekt einen (oder mehrere) Typen, die Zuordnung ist also umgekehrt.

Probleme wie das Russel-Paradoxon können mit Typen gelöst werden.

Kartesisches Produkt

Das kartesische Produkt $A \times B$ (lies: A Kreuz B) von zwei Mengen A, B ist die Menge aller 2-Tupel, die jeweils ein Element von A und ein Element von B enthalten.



Zusammengesetzte Typen: Tupel

Das Äquivalent zum kartesischen Produkt zweier Mengen ist das Produkt zweier Typen, zum Beispiel `int x int`.

Ein Wert dieses Typs enthält zwei Werte vom Typ `int`.

In den meisten Programmiersprachen wird dieses Konzept durch ein Tupel (Eng: `tuple`) gelöst. Ein Tupel erlaubt direkten Zugriff auf den ersten und zweiten Wert. Dies ist jedoch in strikt funktionaler Programmierung nicht erforderlich.

Wichtig: Die Anzahl der Einträge ist immer fest.

Tupel in Python

```
a = tuple(1,2)
a[1] # -> 2

from collections import namedtuple

intxint = namedtuple('intxint', 'first second')
x = intxint(1,2)
x.second # -> 2
|
```

Zusammengesetzte Typen: Union

Die Union (Vereinigung) `Union{A,B}` zweier Typen A und B entspricht der Möglichkeit, dass ein Wert entweder vom Typ A oder vom Typ B ist.

Beispiel: Der Typ `Union{int32, string}` ist entweder ein integer oder ein string.

In den meisten kompilierten Sprachen gibt es Restriktionen für type unions.

Spezielle Typen: Listen

Eine Liste (vector) ist ein Typ, der eine beliebig lange Liste von Werten des gleichen Typs beschreibt.

Eine Liste von int beschreiben wir entweder mit `list(int)` oder mit `[int]`.

Der Unterschied zwischen Tupel und Liste besteht darin, dass ein Tupel eine vorher festgelegte Anzahl an Werten verschiedener Typen enthält. Eine Liste enthält eine unbestimmte Anzahl von Werten des gleichen Typs.

Funktionstypen

Ein Funktionstyp bezeichnet den Eingabe- und den Ausgabotyp einer Funktion. In der Mathematik wird eine Funktion durch Definitions- und Wertebereich beschrieben:

$$f: D \rightarrow Z, x \mapsto y,$$

In der funktionalen Programmierung spezifizieren wir Eingabe- und Ausgabotyp. Wir schreiben `int -> int` für eine Funktion, die ein `int` auf ein `int` abbildet.

Beispiele für Funktionstypen

```
✓ def double(input: int) -> int:  
  |   return 2 * input  
  
✓ def summe(input: List[int]) -> int:  
  |   sum = 0  
✓   |   for i in input:  
    |       sum = sum + i  
    |   return sum
```

Funktionen vs. Methoden, recap

Der wesentliche Unterschied zwischen Funktionen in der objektorientierten Programmierung und der funktionalen Programmierung ist, dass in der funktionalen Programmierung Funktionen

- vollständig
- und stateless, also Seiteneffektfrei sein müssen.

Vollständig bedeutet, dass die Funktionen auf jeder Eingabe eine Ausgabe liefern müssen. Seiteneffektfrei bedeutet, dass ihr Ergebnis nur von der Eingabe, nicht aber vom Kontext abhängen darf.

Beispiel Vollständigkeit

Dieses Verhalten einer Funktion ist zwar technisch korrekt, praktisch müssen wir jedoch im weiteren Verlauf immer wieder überprüfen, ob wir das korrekte Ergebnis erhalten haben.

Dies kann durch Tupel vermieden werden.

```
✓ def incomplete(i: int) -> int:
✓   if i < 0:
      raise ValueError('i is smaller than 0')
      return i

✓ def complete(i: int) -> Union[int, bool]:
✓   if i < 0:
      return False
      return i
```

Beispiel Vollständigkeit

Anstatt einen boolean oder einen int zurückzugeben, geben wir ein Tupel zurück.

Wir können anhand des zweiten Eintrags sehen, ob die Rechnung erfolgreich war oder nicht.

```
def complete2(i: int) -> Tuple(int, bool):  
    if i < 0:  
        return (0, False)  
    return (i, True)
```

Beispiel Seiteneffekte

Eine Funktion ohne Rückgabewert hat immer Seiteneffekte. Dies sollte in der funktionalen Programmierung nie geschehen.

Funktionen ohne Argumente sind erlaubt. Sie dienen als Konstruktoren bestimmter Werte.

```
glob = 5

def inc_glob():
    global glob
    glob = glob + 1

inc_glob()
glob # -> 6
```

Funktionstypen: Higher order functions

Bisher haben wir Funktionen gesehen, die als Eingabe- und Ausgabetypen primitive Typen haben. Ein wesentliches Konzept der funktionalen Programmierung sind jedoch sogenannte “higher order functions”. Dies sind Funktionen, deren Ein- oder Ausgabetypen wiederum Funktionen sind.

Beispiel: Multiplikation

Wir sehen hier eine Funktion, die einen `int` als Eingabe erhält und eine Funktion (`Callable`) zurückgibt. Der Typ der Funktion ist

`int -> (int -> int)`.

```
def multiplyByValue(i : int) -> Callable[[int], int]:  
    return lambda x: x * i  
  
multiplyByValue(2)(3) # -> 6
```

Beispiel: Partial application

```
def setFirstArgument(i: int, f: Callable[[int, int], int]) -> Callable([[int], int]):  
    return lambda x: f(i, x)  
  
def fun(a: int, b: int) -> int:  
    return b / a  
  
divide_by_two = setFirstArgument(2, fun)
```

Partial application: Fallstudie

Partielle Applikation ist insbesondere nützlich, wenn eine Funktion immer wieder mit demselben Argument ausgeführt werden muss. Wir betrachten das Beispiels einer Authentifizierung:

Am Anfang muss der Nutzer seine Login-Informationen in das Programm eingeben. Diese müssen nun in jeder Funktion verwendet werden. Um diesen Vorgang zu vereinfachen, können wir partielle Funktionen definieren, die im Folgenden benutzt werden.

Partial application

```
from typing import NamedTuple

class User(NamedTuple):
    username: str
    password: str

class Data:
    pass

def check_user(user: User) -> bool:
    return True

def do_something_with_data(data: Data) -> Data:
    return Data()

def do_something_prototype(user: User, data: Data) -> tuple(bool, Data):
    checked = check_user(user)
    if checked:
        new_data = do_something_with_data(data)
    else:
        new_data = data
    return (checked, new_data)

user = User('name', 'password')
do_something = lambda x: do_something_prototype(user, x)
```

Partial application

Nachdem der Nutzer eingegeben wurde, können wir unsere Funktionen reduzieren.

Eine voll typisierte Sprache ist an dieser Stelle in der Lage, den Aufruf von `check_user` nur einmal auszuführen, und muss diese Funktion nicht jedes Mal wieder ausführen.

```
from typing import NamedTuple

class User(NamedTuple):
    username: str
    password: str

class Data:
    pass

def check_user(user: User) -> bool:
    return True

def do_something_with_data(data: Data) -> Data:
    return Data()

def do_something_prototype(user: User, data: Data) -> tuple(bool, Data):
    checked = check_user(user)
    if checked:
        new_data = do_something_with_data(data)
    else:
        new_data = data
    return (checked, new_data)

user = User('name', 'password')
do_something = lambda x: do_something_prototype(user, x)
```

Closure

Wenn wir eine partielle Anwendung einer Funktion ausführen, findet eine so genannte Closure statt: In der neuen Funktion ist das partielle Argument für die volle Anwendung der alten Funktion gespeichert.

Closures sind von Programmiersprache zu Programmiersprache unterschiedlich implementiert. Man muss vorsichtig sein, um nicht unerwünschte Seiteneffekte zu erzeugen.

```
x = {}  
foo = lambda y: print(x.keys())  
x['bar'] = 1  
foo(1) # -> dict_keys(['bar'])
```

Currying

Im vorherigen Beispiel haben wir eine Funktion gesehen, welche die Signatur

$$(\text{int } x \ (\text{int } x \ \text{int} \rightarrow \text{int})) \rightarrow (\text{int} \rightarrow \text{int})$$

besitzt. Die partielle Anwendung einer Funktion kann jedoch durch das Currying vereinfacht werden.

Currying

Seien A_1, A_2, A_3, B Typen. Dann kann eine Funktion

$$(A_1 \times A_2 \times A_3 \rightarrow B)$$

äquivalent umgeformt werden in eine Funktion

$$A_1 \rightarrow (A_2 \rightarrow (A_3 \rightarrow B)).$$

Dies bedeutet, dass eine Funktion mit einer Tupeleingabe auch teilweise auf den jeweiligen Daten ausgewertet werden kann. Der Aufruf einer partial application Funktion entfällt.

Beispiel: Currying

```
def sum_nocurry(i: int, j: int) -> int:  
    return i + j  
  
def sum_curry(i: int) -> Callable[[int], int]:  
    return lambda x: i + x  
  
partial_nocurry = lambda x: sum_nocurry(1, x)  
  
partial_curry = sum_curry(1)
```

Beispiel: Currying

Falls eine Funktion oft auf ihren partiellen Aufruf reduziert werden soll, bietet es sich an, das Currying direkt in die Funktion einzubauen. So kann überflüssiger Code vermieden werden.

```
def sum_nocurry(i: int, j: int) -> int:  
    return i + j  
  
def sum_curry(i: int) -> Callable[[int], int]:  
    return lambda x: i + x  
  
partial_nocurry = lambda x: sum_nocurry(1, x)  
  
partial_curry = sum_curry(1)
```

Funktionales Design

Zu einem funktionalen Design zählen sowohl die Auflistung der Typen als auch der Datenfluss.

Zur Erinnerung:

Typen können entweder einzelne primitive Werte, ein Produkt anderer Typen, eine Liste anderer Typen, oder eine Funktion sein. Jedes dieser Beispiele ist wiederum ein Typ.

Funktionales Design: Rock, Paper, Scissors

Wir wollen ein Schere-Stein-Papier-Spiel funktional implementieren.

Hierbei spielt der Spieler gegen den Computer. Der Spieler macht eine Eingabe (“Schere”, “Stein” oder “Papier”), der Computer zufällig wählt eine dieser Eingaben. Anschließend werden die beiden Spielzüge verglichen und das Ergebnis des Spielers (Sieg, Niederlage, Unentschieden) ausgegeben.

Rock Paper Scissors: Spielwert

Als erstes benötigen wir einen Typ, welcher die drei Spielwerte beinhaltet.

Entweder lösen wir dies über Enumerationstypen, oder aber über eine type union.

Ein Enumerationstyp ist ein Typ, für den es eine festgelegte Anzahl an Werten gibt. In unserem Fall sind dies die Werte “Schere”, “Stein”, und “Papier”.

Ein Enum-Typ sorgt dafür, dass auch nur eine festgelegte Anzahl dieser Objekte existiert.

Rock Paper Scissors: Spielwert

Wir können für die Typen die bekannten Klassenobjekte verwenden.

Da in der funktionalen Programmierung niemals Objektmethoden verwendet werden können, benötigen wir auch niemals ein zweites Feld für Methoden.

Spielwert
Stein
Schere
Papier

Rock Paper Scissors: Spielwert

Oft ist es sinnvoll, den Typ direkt mit seiner Dokumentation auszustatten.

Dies hilft dem Leser des Designs, sich über den Zweck des Typen während des Lesens im klaren zu sein

Spielwert
Stein
Schere
Papier

Spielwert der später
verglichen wird.

Rock Paper Scissors

```
import enum

class Spielwert(enum.Enum):
    Stein = "stein"
    Schere = "schere"
    Papier = "papier"
```

Rock Paper Scissors: Spielwert

In python kann jedem enum Wert ein primitiver Wert zugewiesen werden. Wir haben uns hier für die Namen der Spielwerte entschieden.

In anderen Programmiersprachen, zum Beispiel C/C++, wird jedem enum Wert ein nicht-negativer Integer zugeordnet.

```
import enum

class Spielwert(enum.Enum):
    Stein = "stein"
    Schere = "schere"
    Papier = "papier"
```

Rock Paper Scissors: Spielfunktion

Wir definieren die Hauptfunktion des Spiels, indem wir einen weiteren enum Typ definieren, den Ergebniswert. Dieser hat wiederum drei mögliche Werte

- Sieg
- Niederlage
- Unentschieden

Gewinnwert
Sieg
Niederlage
Unentschieden

Rock Paper Scissors

```
class Gewinnwert(enum.Enum):  
    Sieg = "sieg"  
    Niederlage = "niederlage"  
    Unentschieden = "unentschieden"
```

Rock Paper Scissors

Nun können wir unsere Spielfunktion definieren. Wir definieren diese vom Typ
Spielwert x Spielwert -> Gewinnwert.

Die Eingabe der Funktion ist also ein 2-Tupel vom Typ Spielwert x Spielwert, also zwei Spielwerte.

Die Ausgabe ist ein Gewinnwert.

Rock Paper Scissors

Im Design geben wir der Funktion einen Namen und eine Dokumentation.

Es wird an dieser Stelle noch kein Code geschrieben (nächster Schritt).

Es gibt keine spezielle Form, in der die Funktion im Design verankert wird. Wir benutzen lediglich einen Kasten.

Auswertung:
Spielwert x Spielwert -> Gewinnwert

Rock Paper Scissors

Auch hier fügen wir idealerweise wieder direkt eine Dokumentation zur Funktion hinzu. Diese Dokumentation kann einfach angeben, was bei welcher Eingabe zurückgegeben wird.

In der Dokumentation muss insbesondere beschrieben werden, auf welches Argument sich Niederlage und Gewinn beziehen.

Auswertung:
Spielwert x Spielwert -> Gewinnwert

Gibt Gewinn zurück, falls der erste Spielwert den zweiten schlägt. Gibt Niederlage zurück, falls der zweite Spielwert den ersten schlägt. Gibt ansonsten unentschieden zurück.

Rock Paper Scissors

```
def Auswertung(wert1: Spielwert, wert2: Spielwert) -> Gewinnwert:
    if wert1 == wert2:
        return Gewinnwert.Unentschieden
    if (wert1 == Spielwert.Stein and wert2 == Spielwert.Schere) or \
        (wert1 == Spielwert.Schere and wert2 == Spielwert.Papier) or \
        (wert1 == Spielwert.Papier and wert2 == Spielwert.Schere):
        return Gewinnwert.Sieg
    return Gewinnwert.Niederlage
```

Rock Paper Scissors

Um das Problem der zwei Argumente zu vermeiden, können wir auch einen weiteren Typ einführen: Einen Typ Auswertungswert: Spielwert x Spielwert.

In diesem können wir dann die Argumente entsprechend benennen. Wir würden hierfür jedoch eine weitere Funktion benötigen, welche die beiden Werte zusammensetzt, die wiederum das gleiche Problem hätte. Deswegen sehen wir hier davon ab.

Rock Paper Scissors

```
class Auswertungswert(NamedTuple):  
    spieler: Spielwert  
    computer: Spielwert  
  
def ErzeugeAuswertungswert(spieler: Spielwert, computer: Spielwert) -> Auswertungswert:  
    return Auswertungswert(spieler, computer)
```

Rock Paper Scissors: Eingabefunktionen

Wir definieren zwei Eingabefunktionen, eine für den Computer und eine für den Nutzer. Eine erhält einen String (Eingabe des Nutzers), die andere erzeugt einen zufälligen Wert, ohne eine Eingabe.

Den leeren Typ bezeichnen wir mit (), welches ein leeres Tupel andeuten soll.

Rock Paper Scissors: Eingabefunktionen

Wir definieren unsere beiden Eingabefunktionen als

ComputerEingabe: () -> Spielwert

und

SpielerEingabe: () -> Spielwert

Auch wenn beide Funktionen den gleichen Typ haben, müssen wir beide definieren, denn Sie haben unterschiedliche Funktionalität

Rock Paper Scissors: Eingabefunktionen

ComputerEingabe:
() -> Spielwert

SpielerEingabe:
() -> Spielwert

Rock Paper Scissors: Eingabefunktionen

Im strengen Sinne der funktionalen Programmierung sind diese beiden Funktionen nicht zulässig: Ihr Ausgabewert ändert sich unabhängig von der Eingabe.

Es ist jedoch nicht möglich, dies zu vermeiden, wenn man interaktiv arbeiten möchte, also akzeptieren wir dies. Wir vermerken es allerdings in der Dokumentation.

ComputerEingabe:
() -> Spielwert

SpielerEingabe:
() -> Spielwert

Rock Paper Scissors: Eingabefunktionen

ComputerEingabe:
() -> Spielwert

Gibt einen zufällig generierten Spielwert zurück

SpielerEingabe:
() -> Spielwert

Fragt den Nutzer nach Eingabe eines Spielwerts,
und gibt den eingegebenen Wert zurück

Rock Paper Scissors: Eingabefunktionen

```
def ComputerEingabe() -> Spielwert:
    return random.choice(list(Spielwert))

def SpielerEingabe() -> Spielwert:
    string_to_value = {
        Spielwert.Stein.value: Spielwert.Stein,
        Spielwert.Schere.value: Spielwert.Schere,
        Spielwert.Papier.value: Spielwert.Papier,
    }
    print('Wähle schere, stein oder papier')
    return string_to_value[input().strip()]
```

Rock Paper Scissors: Ausgabefunktion

Zuletzt definieren wir die Ausgabefunktion. Diese nimmt den Ausgabewert und schreibt ihn auf das Display. Wir vergeben den Typ

`ZeigeResultat: Gewinnwert -> bool`

wobei der boolsche Ausgabewert der Funktion anzeigt, ob die Ausgabe erfolgreich war.

Man kann den Ausgabewert auch weglassen, allerdings ist es üblich, dass Programme ihr erfolgreiches Terminieren signalisieren.

Rock Paper Scissors: Ausgabefunktion

ZeigeResultat:
Gewinnwert -> bool

Zeigt Gewinn, Niederlage oder Unentschieden an.
Gibt True zurück, wenn die Anzeige erfolgreich war,
sonst False.

Rock Paper Scissors: Ausgabefunktion

```
def ZeigeResultat(resultat: Gewinnwert) -> bool:
    if resultat == Gewinnwert.Unentschieden:
        print("Unentschieden")
    if resultat == Gewinnwert.Sieg:
        print('Gewonnen')
    if resultat == Gewinnwert.Niederlage:
        print('Verloren')
    return True
```

Rock Paper Scissors

Nun gilt es, die einzelnen Teile des Designs zusammen zu fügen.

Zur Erinnerung, dies ist unser bisheriges Design:

Spielwert
Stein
Schere
Papier

Gewinnwert
Sieg
Niederlage
Unentschieden

Auswertung:
Spielwert x Spielwert -> Gewinnwert

ComputerEingabe:
() -> Spielwert

SpielerEingabe:
() -> Spielwert

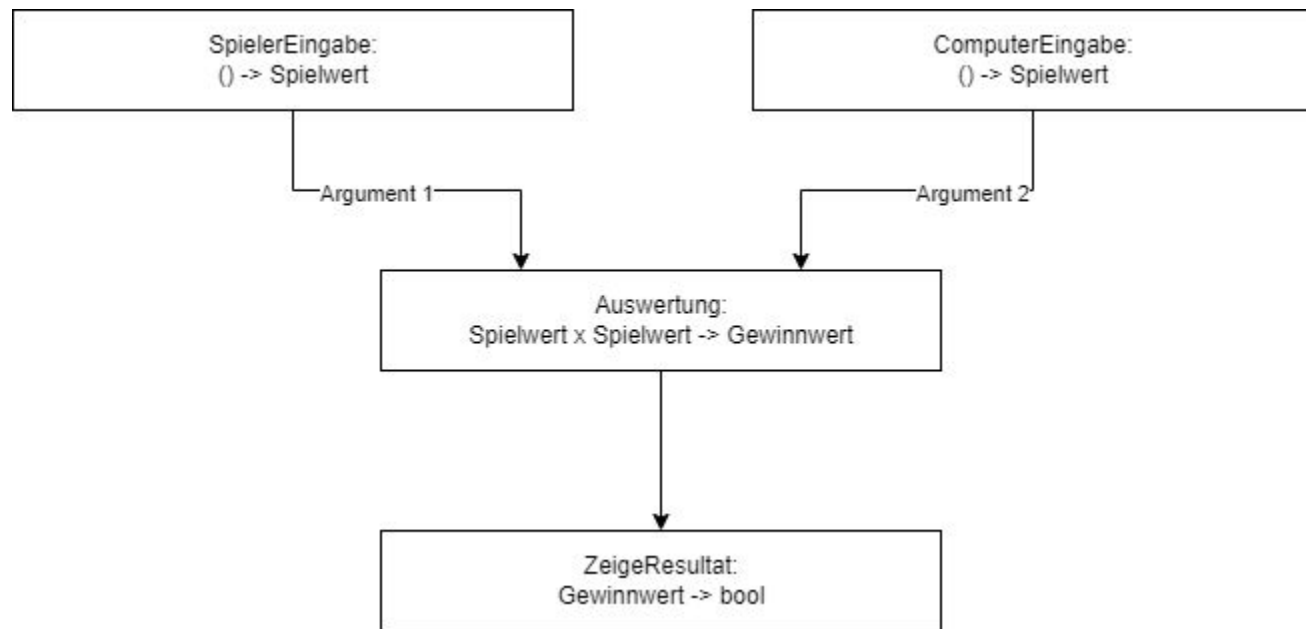
ZeigeResultat:
Gewinnwert -> bool

Rock Paper Scissors

Wir wollen nun die einzelnen Funktionen zum Gesamtprogramm zusammensetzen. Hierzu zeichnen wir einen Abhängigkeitsbaum oder auch Ablaufbaum in unser Diagramm.

Im Baum geben wir an, welche Funktionen in welcher Reihenfolge aufgerufen werden können. Da unser Programm lediglich die Komposition und das kartesische Produkt von Funktionen ist, benötigen wir kaum Zusatzinformationen. Lediglich Argumentreihenfolgen müssen eingetragen werden.

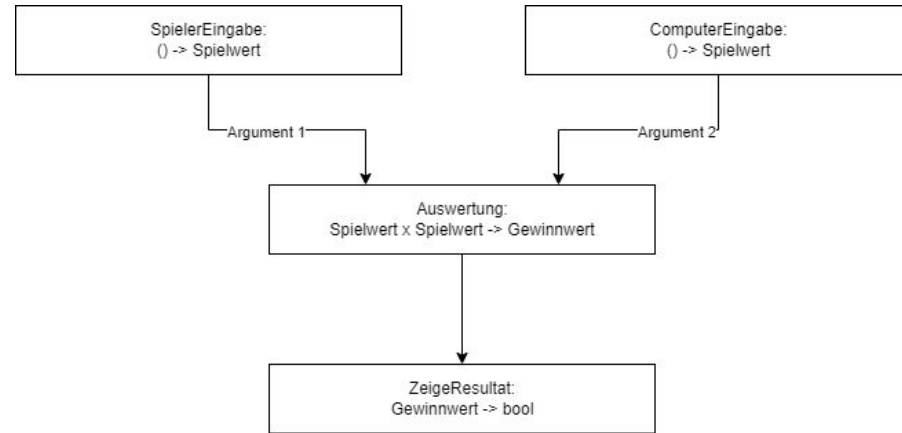
Rock Paper Scissors



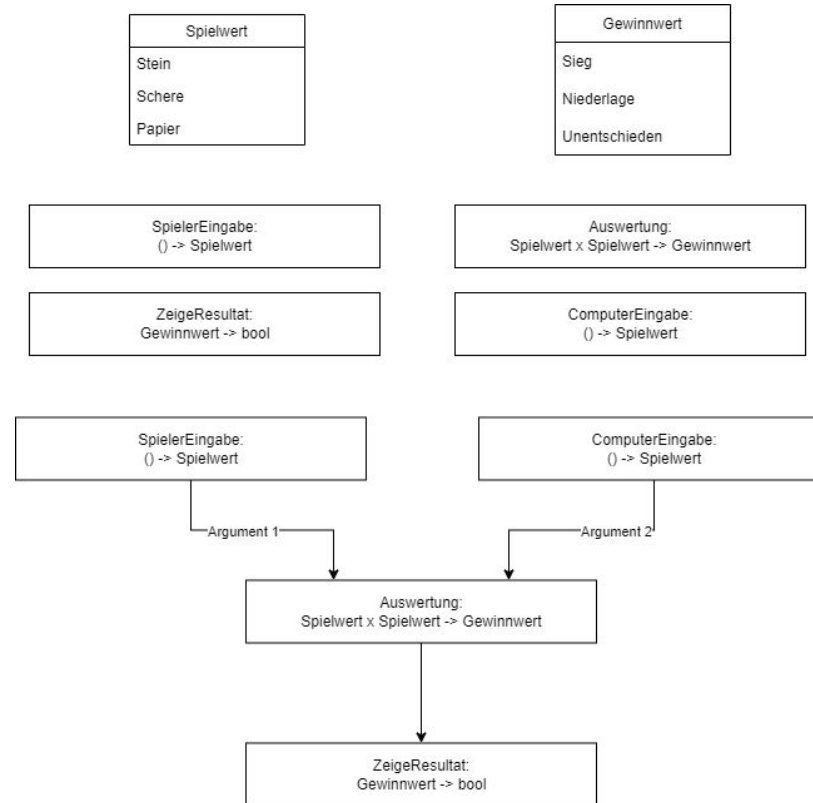
Rock Paper Scissors

Es ist nur dann notwendig, die Argumente zu spezifizieren, wenn dies nicht einwandfrei aus dem Kontext möglich ist.

Hier haben wir zwei Argumente des gleichen Typs, also müssen wir die Reihenfolge spezifizieren.



Rock Paper Scissors



Rock Paper Scissors

```
def RockPaperScissors() -> bool:  
    comp_eingabe = ComputerEingabe()  
    spieler_eingabe = SpielerEingabe()  
    auswertung = Auswertung(spieler_eingabe, comp_eingabe)  
    return ZeigeResultat(auswertung)
```

Rock Paper Scissors

Auf Grund des funktionalen Ansatzes können wir auch das ganze Spiel in einer Zeile spielen:

```
ZeigeResultat(Auswertung(SpielerEingabe(), ComputerEingabe()))
```

Manchmal ist es sogar hilfreich, seinen Code so zu formulieren, damit man mehr Code auf einmal sehen kann.

Erweiterung Typen: Multiple dispatch

Am Anfang des Schere Stein Papier Spiels haben wir angemerkt, dass wir, anstatt eines enums, auch verschiedene Typen für Schere, Stein und Papier definieren können.

In python ist dies nicht sinnvoll, da Funktionsaufrufe, selbst mit unserer Annotation, die Typen ignorieren. In anderen Sprachen können wir uns dies jedoch zunutze machen. Wir werden daher in diesem Abschnitt die Sprache Julia verwenden. Alles hier gezeigte gilt jedoch genauso für C++.

Multiple dispatch

Als multiple dispatch wird die Möglichkeit einer Programmiersprache bezeichnet, Funktionen mit gleichem Namen, aber verschiedenen Eingabetypen zu definieren, welche dann abhängig von den Typen der Eingabe aufgerufen werden.

Dies kann unangenehme if-else Konstrukte in unserer Sprache vermeiden.

Multiple dispatch

```
function f(x::Integer)
    return x + 1
end
```

```
function f(x::Float)
    return x - 1
end
```

```
f(2) # -> 3
```

```
f(2.) # -> 1.
```

Multiple dispatch

Wir haben nun die Möglichkeit, abhängig vom Typ verschiedene Funktionen aufzurufen, welche unterschiedliche Rückgabewerte haben.

Dies können wir uns zunutze machen, um vollständig auf if-else in unseren Programmen zu verzichten.

```
function f(x::Integer)
    return x + 1
end

function f(x::Float)
    return x - 1
end

f(2) # -> 3
f(2.) # -> 1.
```

Multiple dispatch

Im folgenden Beispiel definieren wir unterschiedliche Typen für Schere, Stein, Papier, Gewinn, Niederlage und Unentschieden, und überladen die Funktion Auswertung entsprechend.

Natürlich kann die gleiche Überladung auch für ZeigeResultat stattfinden.

Multiple dispatch

```
struct Schere end
struct Stein end
struct Papier end

struct Gewinn end
struct Niederlage end
struct Unentschieden end
```

```
function Auswertung(x::Schere, y::Papier)
    return Gewinn()
end

Auswertung(::Schere, ::Stein) = Niederlage()
Auswertung(::Stein, ::Papier) = Niederlage()
Auswertung(::Stein, ::Schere) = Gewinn()
Auswertung(::Papier, ::Schere) = Niederlage()
Auswertung(::Papier, ::Stein) = Gewinn()
```

Multiple dispatch

Bei der einzeiligen Schreibweise handelt es sich um eine Kurzschreibweise, welche äquivalent zur langen Definition ist.

Wir sehen weiterhin, dass eine Angabe von Argumentnamen nicht mehr erforderlich ist. Sämtliche notwendigen Entscheidungen werden über den Typ behandelt, somit ist ein Zugriff auf die eigentlichen Objekte nicht mehr notwendig.

```
function Auswertung(x::Schere, y::Papier)
    return Gewinn()
end

Auswertung(::Schere, ::Stein) = Niederlage()
Auswertung(::Stein, ::Papier) = Niederlage()
Auswertung(::Stein, ::Schere) = Gewinn()
Auswertung(::Papier, ::Schere) = Niederlage()
Auswertung(::Papier, ::Stein) = Gewinn()
```

Multiple dispatch

Typisierte Sprachen erlauben auch sogenannte templates. Hier wird anstatt eines Typs eine Variable verwendet:

```
Auswertung(::T, ::T) where {T} = Unentschieden()
```

Hier ist T ein Platzhalter (Template) für einen Typen. Es ist für die Ausführung der Funktion lediglich wichtig, dass der gleiche Typ für beide Argumente verwendet wird.

Multiple dispatch

Wer aufgepasst hat, der merkt, dass wir nun eine neue Signatur für unsere Funktion Auswertung angeben müssen. Diese Signatur ist nun

Auswertung:

`Union[Schere, Stein, Papier]^2 -> Union[Gewinn, Niederlage, Unentschieden]`

Dies ist eine gültige Signatur und kann durch die von uns implementierten Versionen von Auswertung abgedeckt werden.